

Artificial Intelligence in Pla...

Member-only story

I Built a Production RAG System Without LangChain. Here Is How.

LangChain got me to a prototype. Pure Python got me to production.



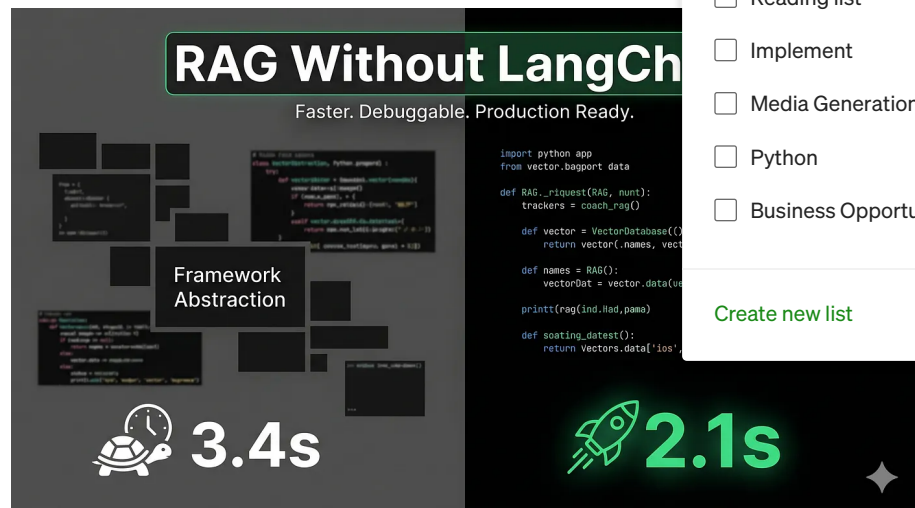
inprogrammer

Follow

6 min read · Apr 14, 2026

104

3



Gemini(nano banana)

- ☐ Reading list
- ☐ Implement
- ☐ Media Generation
- ☐ Python
- ☐ Business Opportunities

Create new list

Friend link for nonmembers- <https://medium.com/@inprogrammer/i-built-a-production-rag-system-without-langchain-here-is-how-15cd5927dfab?sk=e2c8338d0d682a0affff24d6910e5c8f>

Everyone building RAG systems reaches for LangChain first. The documentation points you there. The tutorials use it. The Stack Overflow answers assume it.

I used it too. For three months. Then I replaced it completely.

Not because LangChain is bad. Because I needed to understand exactly what

was happening inside my RAG pipeline, and LangChain's abstractions were standing between me and that understanding....

Every time something went wrong in production, I was debugging LangChain internals instead of my own logic.

So I rebuilt the entire thing in pure Python. Here is exactly what that looks like and what I learned.

What RAG Actually Is

Before the code, a clear picture of what we are building.

RAG stands for Retrieval Augmented Generation. Instead of asking an LLM a question and expecting it to already know the answer from its training data, you first find useful documents from your own data source and then ask the LLM to respond using that information.

There are three steps every time. Convert the query into a vector representation. Find similar vectors in your database. Send the retrieved documents along with the original query to the LLM.

That is the whole system. LangChain organizes these three steps into ready made components. Building without LangChain means you handle each of these steps yourself.

The Stack

I used four libraries. Nothing exotic, nothing experimental.

```
pip install openai pgvector psycpg asyncpg httpx
```

OpenAI for embeddings and the LLM. pgvector for storing and searching vectors inside PostgreSQL. psycpg for database access. httpx for any external API calls the agent needed to make.

No framework. No abstraction layer. Just libraries that do specific things well.

Step 1: Embedding and Storing Documents

Before the system can retrieve anything, it needs to embed and store your documents.

```
import openai
import psycopg
from pgvector.psycopg import register_vector

async def embed_text(text: str) -> list[float]:
    response = await openai.AsyncOpenAI().embeddings.create(
        model="text-embedding-3-small",
        input=text
    )
    return response.data[0].embedding

async def store_document(content: str, metadata: dict, conn):
    embedding = await embed_text(content)
    await conn.execute(
        "INSERT INTO documents (content, metadata, embedding) VALUES (%s, %s, %s)",
        (content, psycopg.types.json.Jsonb(metadata), embedding)
    )
```

The embedding model converts text into a list of numbers that captures semantic meaning. Similar content produces similar vectors. That similarity is what makes retrieval work.

The PostgreSQL setup needs the pgvector extension:

```
CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE documents (
    id SERIAL PRIMARY KEY,
    content TEXT NOT NULL,
    metadata JSONB,
    embedding vector(1536)
);
CREATE INDEX ON documents USING ivfflat (embedding vector_cosine_ops);
```

The index on the embedding column is what makes similarity search fast at scale. Without it, every query scans every row.

Step 2: Retrieving Relevant Documents

When a query comes in, embed it and find the most similar documents.

```
async def retrieve_documents(query: str, conn, limit: int = 5) -> list[dict]:
    query_embedding = await embed_text(query)
```

```

results = await conn.execute(
    """
    SELECT content, metadata,
           1 - (embedding <=> %s::vector) AS similarity
    FROM documents
    ORDER BY embedding <=> %s::vector
    LIMIT %s
    """
    (query_embedding, query_embedding, limit)
)

return [
    {
        "content": row[0],
        "metadata": row[1],
        "similarity": row[2]
    }
    for row in await results.fetchall()
]

```

The `<=>` operator is pgvector's cosine distance operator. Lower distance means higher similarity. The similarity score tells you how relevant each retrieved document is, which you can use to filter out low-quality results.

This is pure SQL. No abstraction layer. When this query is slow, I can add an `EXPLAIN ANALYZE` and see exactly why. When results are poor, I can inspect the similarity scores directly.

Step 3: Generating the Response

Pass the retrieved documents to the LLM with the original query.

```

async def generate_response(query: str, documents: list[dict]) -> str:
    context = "\n\n".join([
        f"Document {i+1} (similarity: {doc['similarity']:.2f}): \n{doc['content']}"
        for i, doc in enumerate(documents)
        if doc['similarity'] > 0.7
    ])

```

```

    if not context:
        return "No relevant documents found for this query."

    messages = [
        {
            "role": "system",
            "content": "Answer questions based only on the provided
context. "
                        "If the context does not contain enough
information, say so clearly."
        },
        {
            "role": "user",
            "content": f"Context:\n{context}\n\nQuestion: {query}"
        }
    ]

    response = await openai.AsyncOpenAI().chat.completions.create(
        model="gpt-4o",
        messages=messages,

```

```

        temperature=0.1
    )
    return response.choices[0].message.content

```

The similarity threshold of 0.7 filters out documents that are not relevant enough. This is something LangChain handles invisibly by default. Building it yourself means you can tune this threshold based on your actual data.

`temperature=0.1` keeps responses consistent and factual. For a RAG system answering questions from a knowledge base, low temperature is almost always the right choice.

Step 4: Putting It Together as a FastAPI Endpoint

```

from fastapi import FastAPI, Depends
from contextlib import asynccontextmanager
import psycpg
from pgvector.psycpg import register_vector

```

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.db = await
    psycpg.AsyncConnection.connect(DATABASE_URL)
    await register_vector(app.state.db)
    yield
    await app.state.db.close()

app = FastAPI(lifespan=lifespan)

async def get_db():
    return app.state.db

@app.post("/query")
async def query_knowledge_base(
    request: QueryRequest,
    db=Depends(get_db)
):
    documents = await retrieve_documents(request.query, db)
    response = await generate_response(request.query, documents)
    return {
        "answer": response,
        "sources": [
            {
                "content": doc["content"][:200],
                "similarity": doc["similarity"],
                "metadata": doc["metadata"]
            }
            for doc in documents
            if doc["similarity"] > 0.7
        ]
    }

```

The endpoint returns both the answer and the sources with similarity scores. This transparency is something I could not get cleanly from LangChain

without significant extra work. Here it is three lines.

What This Approach Costs You

If you are building a RAG system as part of a larger AI agent, I compared four Python agent frameworks here: [I Built a Python AI Agent With 4 Different Frameworks. One Won Clearly.](#)

Pure Python RAG is not free. There are real trade-offs.

You write more code. The LangChain equivalent of this system is fewer lines because the abstractions handle the boilerplate. If development speed is the priority and production debugging is not, LangChain wins on that dimension.

You lose the ecosystem. LangChain has integrations for dozens of vector databases, dozens of LLMs, and dozens of document loaders. Building without it means you are responsible for any integration you need.

You own every bug. When something breaks, it is your code that broke. No framework to blame and no community issue tracker to check.

What You Gain

Every line of this system is code I wrote and understand completely. When the similarity search returns poor results, I know exactly where to look. When response quality drops, I can inspect every step of the pipeline independently.

Production debugging went from hours to minutes after the switch. Not because the bugs changed but because finding them stopped requiring archaeology through framework internals.

Performance also improved. Removing the LangChain abstraction layer reduced average response time from 3.4 seconds to 2.1 seconds on the same queries. The overhead of framework abstractions is real.

The system has been running in production for two months with a 97% success rate on first attempt. No LangChain version breaking changes, no mysterious failures from framework updates, no abstraction layer hiding what is actually happening.

When to Use Each Approach

Build without a framework when you need production reliability, full debuggability, and are comfortable writing the integration code yourself. This approach works best for focused RAG systems with clear requirements.

Use LangChain when you are prototyping, exploring capabilities, or need rapid integration with many different data sources. The ecosystem advantage is real for exploration.

The mistake is using LangChain in production because you used it in prototyping. The requirements are different enough that it is worth evaluating the trade-offs before committing.

CTA

If this helped you think differently about your RAG system architecture, follow me here on Medium. I write about real Python and AI production problems with exact fixes and actual numbers, not theory.

Building a RAG system and hitting a problem that was not covered here? Drop it in the comments. I read every one.

A message from our Founder

Hey, Sunil here. I wanted to take a moment to thank you for reading until the end and for being a part of this community. Did you know that our team run these publications as a volunteer effort to over 3.5m monthly readers? We don't receive any funding, we do this to support the community.

If you want to show some love, please take a moment to follow me on LinkedIn, TikTok, Instagram. You can also subscribe to our weekly newsletter. And before you go, don't forget to clap and follow the writer!

[Python](#)[Artificial Intelligence](#)[Software Development](#)[Programming](#)[Fastapi](#)



Published in Artificial Intelligence in Plain English

43K followers · Last published 2 hours ago

Follow

New AI, ML and Data Science articles every day. Follow to join our 3.5M+ monthly readers.



Written by inprogrammer

1.3K followers · 15 following

Follow

I break Python & FastAPI apps in production... then fix them. Real bugs, performance issues, and solutions that actually work.

